

Sistemi Operativi (Modulo II)

Capitolo 05 — Gestione della Memoria

Luca Argentino

A.A. 2025–2026

Contents

5	Gestione della Memoria	3
5.1	Introduzione: il Memory Manager	3
5.2	Binding, Loading e Linking	4
5.2.1	Binding durante la Compilazione	4
5.2.2	Binding durante il Caricamento	5
5.2.3	Binding durante l'Esecuzione	5
5.2.4	Indirizzi Logici e Indirizzi Fisici	6
5.2.5	Loading Dinamico e Linking Dinamico	9
5.3	Allocazione memoria	10
5.3.1	Allocazione a Partizioni Fisse	10
5.3.2	Allocazione a Partizioni Dinamiche	12
5.4	Compattazione	14
5.5	Strutture Dati per la Gestione della Memoria Libera (con allocazione dinamica)	15
5.5.1	Mappa di Bit (Bitmap)	15
5.5.2	Lista con Puntatori	15
5.5.3	Algoritmi di Selezione del Blocco Libero	17
5.6	Paginazione	18
5.6.1	Funzionamento: Esempio con Multiprogrammazione	18
5.6.2	Traduzione degli Indirizzi con la MMU	19
5.6.3	Dimensione delle Pagine: il Trade-off	20
5.6.4	Implementazione della Page Table	20
5.6.5	Translation Lookaside Buffer (TLB)	20
5.7	Segmentazione	21
5.7.1	Segmentazione vs. Paginazione	22
5.8	Segmentazione + Paginazione	22
5.9	Memoria Virtuale	23
5.9.1	Motivazione	23
5.9.2	Implementazione Memoria Virtuale	23
5.9.3	Implementazione: Demand Paging (Paginazione a Richiesta)	24
5.9.4	Gestione Dettagliata di un Page Fault	26
5.10	Algoritmi di Rimpiazzamento	28
5.10.1	Algoritmo FIFO	29
5.10.2	Algoritmo Ottimale (MIN / OPT)	31
5.10.3	Algoritmo LRU (Least Recently Used)	31
5.10.4	Least Frequently Used (LFU)	39
5.11	Allocazione dei Frame, Thrashing e Working Set	40

5.11.1 Strategie di Allocazione dei Frame (memoria virtuale)	40
5.11.2 Thrashing	40
5.11.3 Working Set	41
Riepilogo	42

Capitolo 5

Gestione della Memoria

5.1 Introduzione: il Memory Manager

Un calcolatore mette a disposizione dei programmi due tipi di memoria: la **memoria principale** (RAM), veloce ma volatile e limitata, e la **memoria secondaria** (disco), lenta ma persistente e capiente.

Il sistema operativo deve gestire la memoria principale in modo che piu' processi possano coesistere ed eseguire in modo efficiente.

La componente del kernel deputata a questo compito e' il **memory manager** (MM). E' fondamentale distinguere subito due acronimi simili ma profondamente diversi:

- **MM – Memory Manager:** componente *software* del kernel. Tiene traccia di quali zone di memoria sono libere e quali occupate; assegna memoria ai processi e la rilascia al termine.
- **MMU – Memory Management Unit:** componente *hardware* tipicamente integrata nella CPU. Traduce in tempo reale gli indirizzi generati dal processore (indirizzi *logici*) negli indirizzi effettivi della RAM (indirizzi *fisici*).

Nota – Prospettiva storica

I meccanismi che studieremo vanno dal piu' semplice al piu' sofisticato. Alcuni potrebbero sembrare obsoleti, ma nell'informatica la storia si ripete: allocazione a partizioni fisse, binding a compile-time e sistemi monoprogrammati sono ancora in uso nei sistemi *embedded*, nei microcontrollori e nelle smart-card, dove la semplicita' e la prevedibilita' del comportamento sono prioritarie rispetto alle prestazioni assolute.

5.2 Binding, Loading e Linking

Definizione – Binding

Il **binding** e' l'operazione di *associazione* tra indirizzi logici (nomi simbolici come variabili, label, nomi di funzione) e indirizzi fisici della memoria. In sostanza, risponde alla domanda: “a quale cella di RAM corrisponde questa variabile?”

Il binding puo' avvenire in tre momenti distinti.

5.2.1 Binding durante la Compilazione

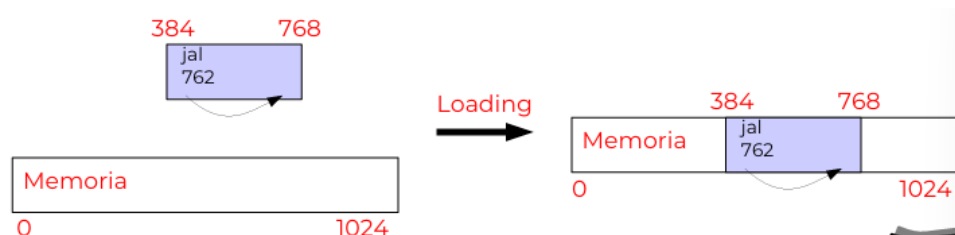


Figure 5.1: Binding a compile-time. Il compilatore calcola gli indirizzi fisici definitivi e li iscrive direttamente nel codice generato. Il programma viene caricato sempre allo stesso indirizzo fisso in memoria.

Il compilatore conosce (o decide) gia' in fase di traduzione dove il programma sara' caricato in memoria. Gli indirizzi vengono calcolati staticamente e inseriti nel codice oggetto: il risultato si chiama **codice assoluto**.

Nell'esempio, la label `jal 384` (offset relativo nel codice sorgente) viene risolta direttamente come `jal 762` (indirizzo fisso nella RAM). Il codice puo' essere caricato *solo* all'indirizzo per cui e' stato compilato: se quella zona e' occupata da un altro processo, il programma non puo' essere eseguito.

Vantaggi.

- Nessun hardware speciale richiesto (niente MMU).
- Veloce: nessuna traduzione a run-time.
- Semplice: il programma vede direttamente la memoria fisica.

Svantaggi.

- **Non funziona con la multiprogrammazione:** se due programmi sono compilati per lo stesso indirizzo base, non possono coesistere in memoria.
- Non flessibile: la ricompilazione e' necessaria per cambiare la posizione in memoria.

Utilizzi attuali. Codice per microcontrollori (indirizzo di partenza fisso nella ROM), kernel monolitico, file `.COM` di MS-DOS.

5.2.2 Binding durante il Caricamento

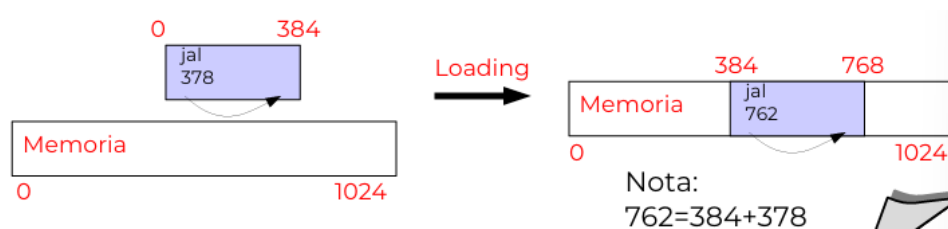


Figure 5.2: Binding a load-time. Il compilatore produce codice rilocabile con indirizzi relativi (offset dalla base). Al momento del caricamento, il loader aggiusta tutti gli indirizzi sommando la base B scelta: nell'esempio $762 = 384 + 378$.

Il compilatore produce **codice rilocabile**: invece di indirizzi assoluti, inserisce *offset relativi* all'indirizzo base (o al Program Counter). Il codice oggetto contiene una *tabella di rilocalizzazione* che indica quali parole devono essere aggiustate.

Al momento del caricamento, il **loader** sceglie un indirizzo base B (dove la memoria e' libera) e somma B a tutti gli indirizzi rilocabili. Nell'esempio della slide: offset 378, base $B = 384$, quindi l'indirizzo fisico diventa $384 + 378 = 762$.

Vantaggi.

- Permette la multiprogrammazione: ogni processo puo' essere caricato in una zona diversa.
- Nessun hardware speciale a run-time.

Svantaggi.

- Una volta caricato, il processo *non puo' essere spostato*: se si vuole rilocare il processo (compattazione), bisogna fermare l'esecuzione e modificare tutto il codice in memoria.
- Richiede formati di file eseguibile piu' complessi (sezione di rilocalizzazione).

5.2.3 Binding durante l'Esecuzione

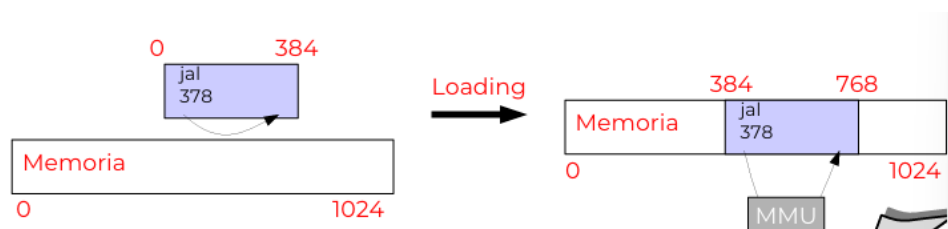


Figure 5.3: Binding a run-time. Il codice rimane rilocabile; la MMU traduce ogni indirizzo logico in fisico *on-the-fly* durante l'esecuzione. Il loader carica il programma come prima, ma non modifica gli indirizzi: ci pensa la MMU hardware.

E' il meccanismo adottato da tutti i sistemi operativi moderni. Il codice rimane rilocabile (indirizzi relativi), ma la traduzione da logico a fisico avviene *ad ogni accesso in memoria*, in hardware, tramite la **MMU**.

Questo schema offre la massima flessibilit :

- Il processo puo' essere spostato fisicamente in memoria anche durante l'esecuzione (il codice non deve essere modificato, solo i registri della MMU vengono aggiornati).
- La stessa area di memoria virtuale puo' mappare su frame fisici non contigui (*paginazione*).
- E' possibile realizzare la *memoria virtuale*: alcune pagine possono stare su disco e venire caricate solo quando necessario.

5.2.4 Indirizzi Logici e Indirizzi Fisici

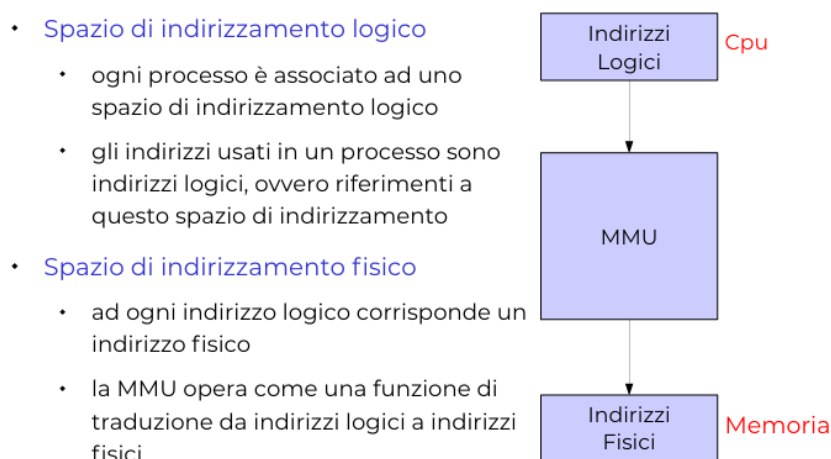


Figure 5.4: Separazione tra spazio di indirizzamento logico (visto dal processo) e spazio di indirizzamento fisico (RAM reale). La CPU genera indirizzi logici; la MMU li converte in indirizzi fisici prima di accedere alla memoria.

Definizione

Ogni processo vive in uno **spazio di indirizzamento logico** privato. Tutti gli indirizzi che il processore genera (load, store, branch) sono **indirizzi logici**, relativi a questo spazio virtuale. La MMU li converte in **indirizzi fisici** prima di inviarli al bus di memoria.

Questo livello di indirizzamento   fondamentale: ogni processo crede di avere accesso esclusivo a una memoria privata a partire dall'indirizzo 0, ma in realt  i suoi dati sono sparsi nella RAM fisica insieme a quelli degli altri processi.

MMU con Registro di Rilocalizzazione

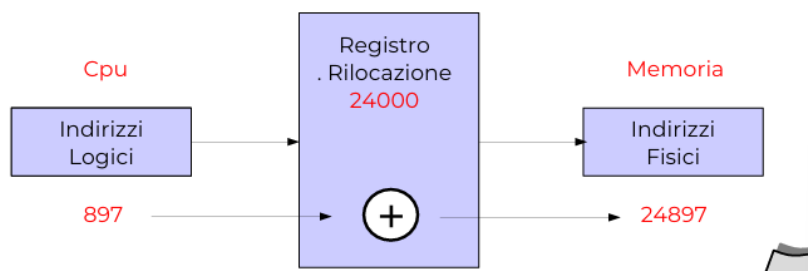


Figure 5.5: MMU basata su registro di rilocalizzazione (valore 24000). L'indirizzo logico 897 viene sommato al registro per ottenere l'indirizzo fisico 24897. Tutta la traduzione avviene in hardware in un singolo ciclo di clock.

La MMU più semplice utilizza un **registro di rilocalizzazione** (o *base register*). Se il registro contiene il valore R , ogni indirizzo logico l viene tradotto in $l + R$. Lo spazio logico $0 \dots \text{MAX}$ diventa lo spazio fisico $R \dots R + \text{MAX}$.

Il kernel carica nel registro di rilocalizzazione l'indirizzo fisico di partenza del processo corrente ogni volta che effettua un context switch. Nei processori Intel 80x86 i registri CS, DS, SS, ES svolgono proprio questo ruolo per i quattro segmenti principali.

MMU con Registro di Rilocalizzazione e Limite

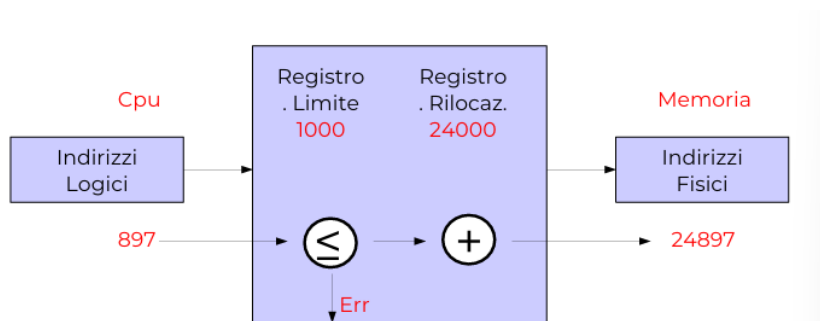


Figure 5.6: MMU con registro di rilocalizzazione (24000) e registro limite (1000). Prima della traduzione si verifica che l'indirizzo logico (897) sia minore del limite (1000, ovvero che i processi che vogliono accedere abbiano memoria $\leq$ a quella accordata): condizione soddisfatta, quindi si procede con la somma. Se l'indirizzo fosse ≥ 1000 , la MMU genererebbe un errore di protezione.

Per implementare la **protezione della memoria** si aggiunge un **registro limite** (o *bound register*). Prima della traduzione, la MMU verifica:

$$\text{indirizzo logico} < \text{registro limite}$$

Se la condizione non è rispettata, viene generato un *trap* hardware (che il kernel cattura e segnala al processo come *Segmentation Fault*). Questo impedisce a un processo di leggere o scrivere la memoria di un altro processo o del kernel.

Nota – Perché due registri?

Il registro di rilocazione risponde alla domanda “dove si trova il processo in memoria?”. Il registro limite risponde a “quanto è grande?”. Insieme, definiscono l'intervallo fisico $[R, R + L)$ di competenza del processo. Il kernel li aggiorna a ogni context switch con i valori del nuovo processo in esecuzione.

Esempio – Il trucco del Context Switch: Processo A vs Processo B

Entrambi i processi, A e B, nel loro codice compilato chiedono di leggere la variabile al loro indirizzo logico 100. Vediamo come il Sistema Operativo (S.O.) e la MMU evitano che si sovrascrivano a vicenda nella RAM.

Fase 1: Esecuzione del Processo A

- Il S.O. ha precedentemente caricato il Processo A nella RAM fisica a partire dall'indirizzo 20000.
- Prima di cedere la CPU al Processo A, il S.O. imposta il **Registro di Rilocazione della MMU = 20000**.
- Il Processo A va in esecuzione e richiede: `LOAD 100`.
- La MMU traduce in hardware: $20000 + 100 = \mathbf{20100}$ (Indirizzo Fisico reale).

Fase 2: Context Switch (Cambio di contesto)

- Il tempo a disposizione scade (o avviene un interrupt). Il S.O. congela il Processo A per far girare il Processo B.

Fase 3: Esecuzione del Processo B

- Il S.O. ha caricato il Processo B in un'altra zona libera della RAM, ad esempio all'indirizzo 80000.
- *Azione chiave:* Il S.O. sovrascrive il **Registro di Rilocazione della MMU = 80000**.
- Il Processo B va in esecuzione e richiede la sua istruzione: `LOAD 100`.
- La MMU traduce al volo: $80000 + 100 = \mathbf{80100}$ (Indirizzo Fisico reale).

Risultato: Pur avendo richiesto al processore lo stesso identico indirizzo logico (100), la traduzione hardware basata sul contesto ha dirottato le richieste su celle di RAM fisica completamente diverse (20100 e 80100). L'isolamento è garantito.

5.2.5 Loading Dinamico e Linking Dinamico

Queste sono tecniche per migliorare le prestazioni e ridurre l'uso di memoria.

Loading dinamico. Alcune routine (tipicamente gestori di errori rari) non vengono caricate all'avvio del processo, ma rimangono su disco in formato rilocabile. Vengono caricate in memoria solo la prima volta che vengono invocate. Spetta al programmatore sfruttare questa possibilità (il S.O. fornisce le API di supporto, come `dlopen` su Linux).

Linking statico vs. dinamico. Con il **linking statico**, il linker incorpora una copia completa di ogni libreria nell'eseguibile finale: se 100 processi usano la `printf`, ci sono 100 copie di `printf` in memoria. Con il **linking dinamico** (*shared libraries*, file `.so` su Linux o `.dll` su Windows), esiste una sola copia della libreria in memoria, condivisa da tutti i processi. Il codice di libreria è *reentrant* (senza stato globale modificabile), in modo che più processi possano eseguirlo simultaneamente.

Esempio – Vantaggi del linking dinamico

- **Risparmio di memoria:** una sola copia di `libc.so` per tutti i processi. Permette così di avere eseguibili compatti
- **Aggiornamento automatico:** aggiornare la libreria su disco aggiorna tutti i programmi che la usano alla successiva esecuzione, senza ricompilazione.
- **Plug-in:** con `dlopen()` è possibile caricare moduli aggiuntivi a run-time (browser, IDE, giochi).

Svantaggi: può causare problemi di "versioning"

occorre aggiornare le versioni solo se non sono incompatibili, cambiamento numero di revisione e non di release

5.3 Allocazione memoria

Definizione – Allocazione

Allocare memoria significa reperire e assegnare una zona di RAM a un processo. Si dice **contigua** se tutta la memoria assegnata forma un unico blocco di celle consecutive. Si dice **non contigua** se il processo puo' ricevere aree separate (come nella paginazione).

5.3.1 Allocazione a Partizioni Fisse



Figure 5.7: Memoria suddivisa in tre partizioni fisse di dimensione diversa (320K, 512K, 768K). Il Sistema Operativo risiede nella parte bassa (indirizzo 0). Ogni processo viene caricato nella prima partizione libera abbastanza grande.

E' il meccanismo piu' semplice: la memoria viene suddivisa *a priori* in un numero fisso di **partizioni** di dimensione predefinita. Ogni processo viene caricato in una partizione libera abbastanza grande da contenerlo.

La gestione puo' avvenire con:

- **Una coda per partizione:** i processi aspettano nella coda della partizione di dimensione adatta. Rischio: code lunghe per partizioni piccole e partizioni grandi vuote.
- **Coda comune:** lo scheduler sceglie, tra i processi in attesa, quello piu' adatto alla partizione libera disponibile.

Sistemi monoprogrammati. Caso limite con una sola partizione: un unico processo utente occupa tutta la memoria disponibile. Era il modello di MS-DOS ed e' ancora usato in sistemi embedded semplici.



Figure 5.8: Frammentazione interna. La partizione 2 (da 512K a 768K) e' assegnata a un processo che occupa solo parte dello spazio disponibile. Lo spazio inutilizzato (area grigia/bianca all'interno della partizione) e' sprecato e non puo' essere usato da altri processi finche' questo non termina.

Definizione – Frammentazione Interna

Si parla di **frammentazione interna** quando l'unità di allocazione è più grande della quantità di memoria effettivamente richiesta dal processo. Lo spazio “interno” all'unità allocata ma non usato è irrecuperabile finché il processo non termina. Non è un problema esclusivo delle partizioni fisse: si presenta in tutti i sistemi che allocano in unità di dimensione minima (come l'ultima pagina nella paginazione).

5.3.2 Allocazione a Partizioni Dinamiche

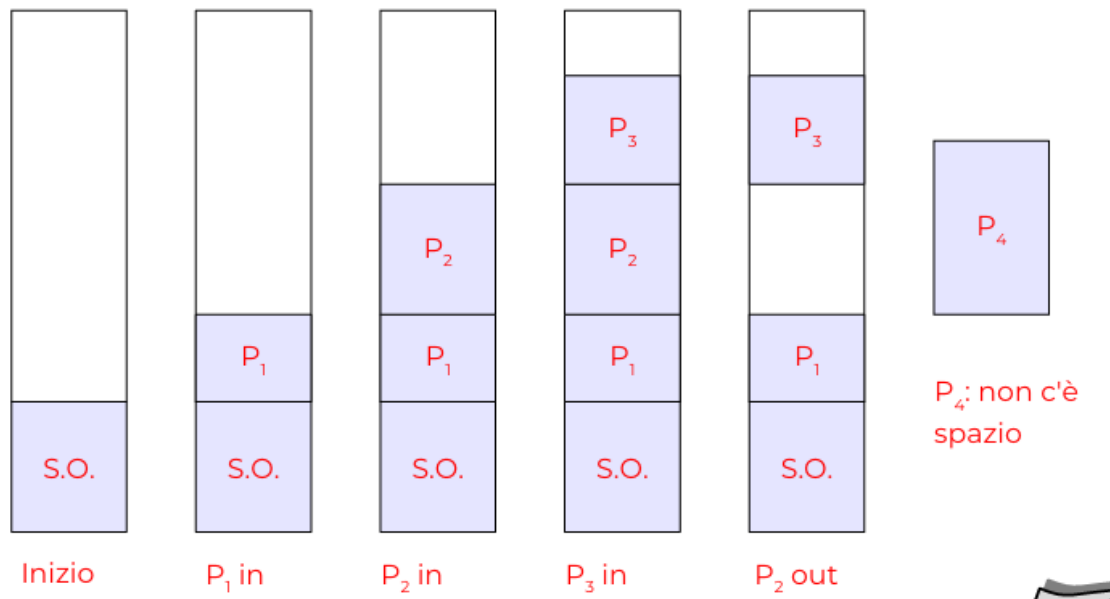


Figure 5.9: Evoluzione della memoria con partizioni dinamiche. Si leggono le colonne da sinistra a destra: (1) situazione iniziale; (2) entrano P1, P2, P3; (3) P2 termina, lascia un buco; (4) P4 vuole entrare ma nessun buco singolo e' abbastanza grande. Questo e' il problema della frammentazione esterna.

Con le partizioni dinamiche, il kernel assegna *esattamente* la quantita' di memoria richiesta da ogni processo (**niente sovra-allocazione**). Man mano che i processi terminano, lasciano dei **buchi** (*hole*) nella mappa di memoria.

Nell'esempio della figura 5.9 si vede chiaramente l'evoluzione:

1. All'inizio la memoria contiene solo il S.O.
2. Entrano P1 (600K), P2 (500K), P3 (250K): la memoria si riempie dal basso verso l'alto.
3. P2 termina: si crea un buco di 500K dove stava P2.
4. P4 vuole 450K: il buco di P2 non e' abbastanza grande, e i due buchi separati (quello sopra e quello sotto P3) non sono contigui. P4 non puo' entrare nonostante la memoria libera totale sia sufficiente.

Definizione – Frammentazione Esterna

La **frammentazione esterna** si verifica quando la memoria libera totale e' sufficiente per un nuovo processo, ma e' suddivisa in tanti piccoli buchi non contigui che nessuno da solo e' abbastanza grande. E' una conseguenza del susseguirsi di allocazioni e deallocazioni di dimensioni diverse.

Nota – Origine della Frammentazione in Memoria

Nella gestione della memoria dei sistemi operativi, il fenomeno della frammentazione si divide in due tipologie principali, ciascuna derivante da diverse strategie di allocazione dello spazio.

Frammentazione Interna

La **frammentazione interna** deriva dall'utilizzo di tecniche di *allocazione a partizioni fisse* o dalla *paginazione*.

- **Causa principale:** La memoria centrale viene suddivisa in blocchi o "pagine" di dimensione fissa (ad esempio, 4 KB). Quando un processo richiede una quantità di memoria che non è un multiplo esatto della dimensione del blocco, l'ultimo blocco assegnato non viene riempito completamente.
- **Risultato:** Lo spazio inutilizzato all'interno del blocco allocato viene sprecato, in quanto il sistema operativo lo considera occupato e non può assegnarlo a nessun altro processo.

Frammentazione Esterna

La **frammentazione esterna** deriva dall'utilizzo di tecniche di *allocazione a partizioni dinamiche* o dalla *segmentazione*.

- **Causa principale:** I processi vengono caricati e rimossi dalla memoria in momenti diversi, occupando blocchi di dimensione variabile in base alle loro specifiche necessità. Con il passare del tempo, questo continuo turnover crea numerosi "buchi" di memoria libera sparsi in modo irregolare per tutta la RAM.
- **Risultato:** Sebbene la somma totale dello spazio libero sia potenzialmente sufficiente per soddisfare una nuova richiesta di allocazione, tale spazio non è contiguo. Di conseguenza, il sistema non trova un singolo blocco abbastanza grande e non può caricare il nuovo processo.

5.4 Compattazione

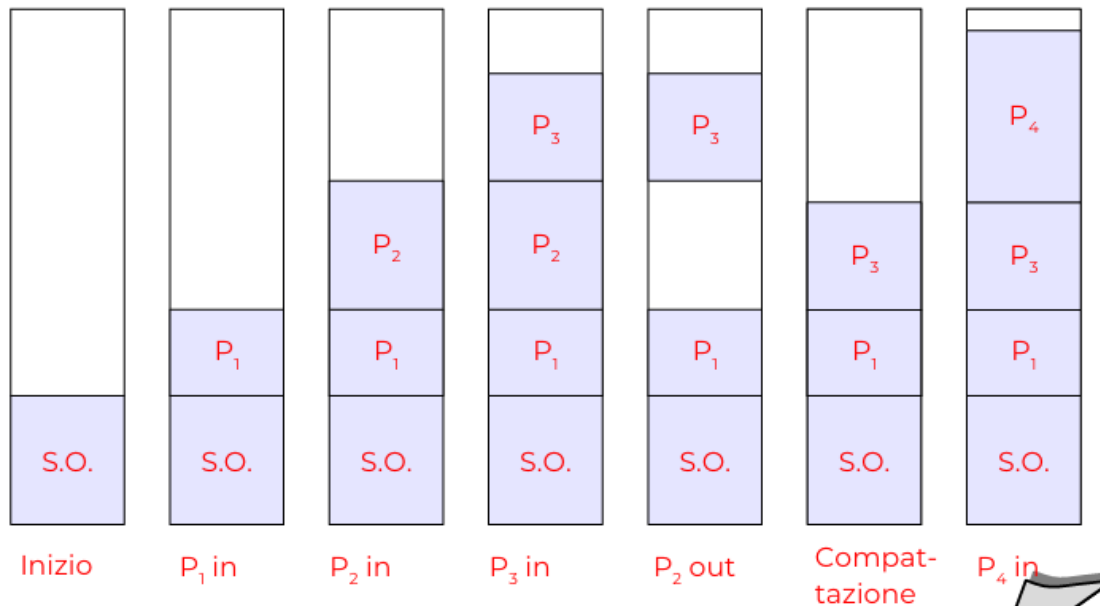


Figure 5.10: Compattazione della memoria. Le ultime due colonne mostrano il risultato: tutti i processi vengono spostati verso il basso, riunendo tutti i buchi in un unico grande blocco libero in cima. Dopo la compattazione P4 può finalmente essere caricato.

La **compattazione** risolve la frammentazione esterna: tutti i processi vengono spostati verso un estremo della memoria, riunendo tutti i buchi in un unico grande blocco contiguo.

Nota – Perché la compattazione è costosa

- Bisogna **copiare fisicamente** i dati di tutti i processi in memoria: operazione $O(n)$ sulla dimensione totale della RAM occupata.
- I processi devono essere **fermi** durante lo spostamento per evitare accessi a indirizzi invalidi.
- Richiede che il binding sia a run-time (con MMU): altrimenti gli indirizzi nei programmi diventerebbero tutti sbagliati dopo lo spostamento.
- Incompatibile con sistemi interattivi ad alta reattività.

Nella pratica moderna la compattazione non si usa mai: il problema della frammentazione esterna viene risolto alla radice con la **paginazione**.

5.5 Strutture Dati per la Gestione della Memoria Libera (con allocazione dinamica)

Il kernel deve tenere traccia di quali zone di memoria sono libere e quali occupate. Le due strutture dati principali sono la **mappa di bit** e la **lista con puntatori**.

5.5.1 Mappa di Bit (Bitmap)

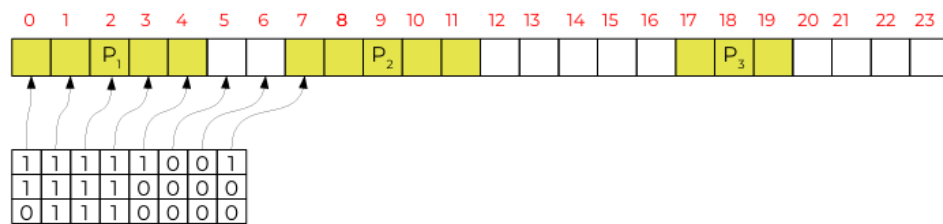


Figure 5.11: Mappa di bit.

La memoria viene suddivisa in **unità di allocazione** numerate (0, 1, 2, ...) di dimensione fissa (es. 4 byte, 1 KB). Per ogni unità c'è un bit nella bitmap: 1 = occupata, 0 = libera.

I processi P_1 , P_2 , P_3 occupano blocchi contigui visibili come sequenze di 1 nella bitmap.

Trade-off sulla dimensione dell'unità:

- Unità piccole $\Rightarrow$ bitmap grande, ma frammentazione interna trascurabile.
- Unità grandi $\Rightarrow$ bitmap piccola, ma maggiore frammentazione interna.

Vantaggi: dimensione della bitmap fissa e calcolabile a priori ($\frac{RAM}{\text{dim. unità} \times 8}$ byte). Struttura semplice e compatta.

Svantaggi: per allocare k unità consecutive bisogna cercare k bit a 0 consecutivi, operazione $O(m)$ con m = numero totale di unità.

5.5.2 Lista con Puntatori

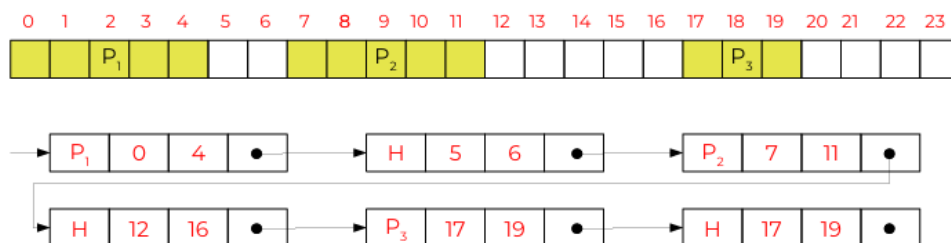


Figure 5.12: Lista con puntatori. La lista concatenata permette di trovare efficientemente i blocchi adiacenti.

Si mantiene una lista concatenata di blocchi. Ogni nodo contiene:

- Tipo: **P** (processo) o **H** (hole, buco libero).
- Indirizzo di inizio e fine (o inizio + dimensione).
- puntatore al prossimo nodo

Allocazione. Si cerca nella lista un blocco H abbastanza grande, lo si divide in due: la prima parte diventa un blocco P della dimensione richiesta, il resto rimane H con dimensione ridotta.

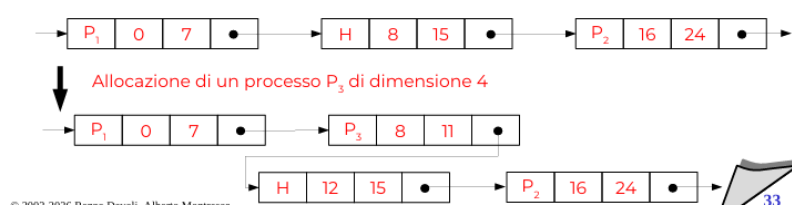


Figure 5.13: Allocazione processo con dimensione inferiore al blocco libero disponibile

Deallocazione (quattro casi). Quando un processo P1 termina, il suo blocco diventa H. Si guarda cosa c'è prima e dopo nella lista:

1. P1 è tra due P $\Rightarrow$ si crea un nuovo H isolato.
2. P1 è a sinistra di un H $\Rightarrow$ si fondono: il nuovo H va da inizio(P1) a fine(H_destro).
3. P1 è a destra di un H $\Rightarrow$ si fondono: il nuovo H va da inizio(H_sinistro) a fine(P1).
4. P1 è tra due H $\Rightarrow$ i tre blocchi si fondono in un unico grande H.

La fusione è $O(1)$ con una lista doppiamente collegata: si guardano solo i nodi adiacenti.

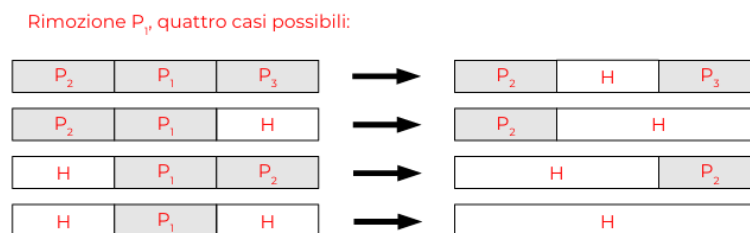


Figure 5.14: Deallocazione memoria processo

5.5.3 Algoritmi di Selezione del Blocco Libero

- **First Fit:** si scorre la lista dall'inizio e si prende il *primo* blocco abbastanza grande. Veloce; tende a frammentare la parte bassa della memoria ma lascia blocchi grandi in quella alta. In pratica e' il piu' efficiente.
- **Next Fit:** come First Fit ma si riparte dal punto dell'ultima allocazione (puntatore circolare). Distribuisce le allocazioni ma ha **prestazioni peggiori** di First Fit nella pratica.
- **Best Fit:** si esamina tutta la lista e si sceglie il blocco libero piu' piccolo che sia ancora sufficiente. Minimizza lo spreco ma e' piu' lento ($O(n)$) e lascia buchi residui minuscoli $\Rightarrow$ piu' frammentazione di First Fit.
- **Worst Fit:** si sceglie il blocco piu' grande. L'idea e' lasciare residui abbastanza grandi da essere riutilizzati, ma in pratica peggiora la situazione perche' consuma i blocchi grandi.

5.6 Paginazione

Definizione – Paginazione

La **paginazione** e' la soluzione moderna alla frammentazione esterna (la minimizza, possiamo dire "elimina") e riduce la frammentazione interna. Si basa sull'idea di spezzare sia la memoria logica sia quella fisica in blocchi di dimensione fissa:

- Lo spazio logico di un processo e' diviso in blocchi di dimensione fissa chiamate **pagine** (*pages*).
- La memoria fisica e' divisa in insiemi di blocchi chiamati **frame** (*page frames*).
- Pagine e frame hanno la *stessa* dimensione fissa.
- I frame vengono assegnati ai processi in qualunque posizione fisica. **basta che contengano tutte le pagine del processo.**
- La corrispondenza pagina → frame e' mantenuta nella **page table** del processo.

5.6.1 Funzionamento: Esempio con Multiprogrammazione

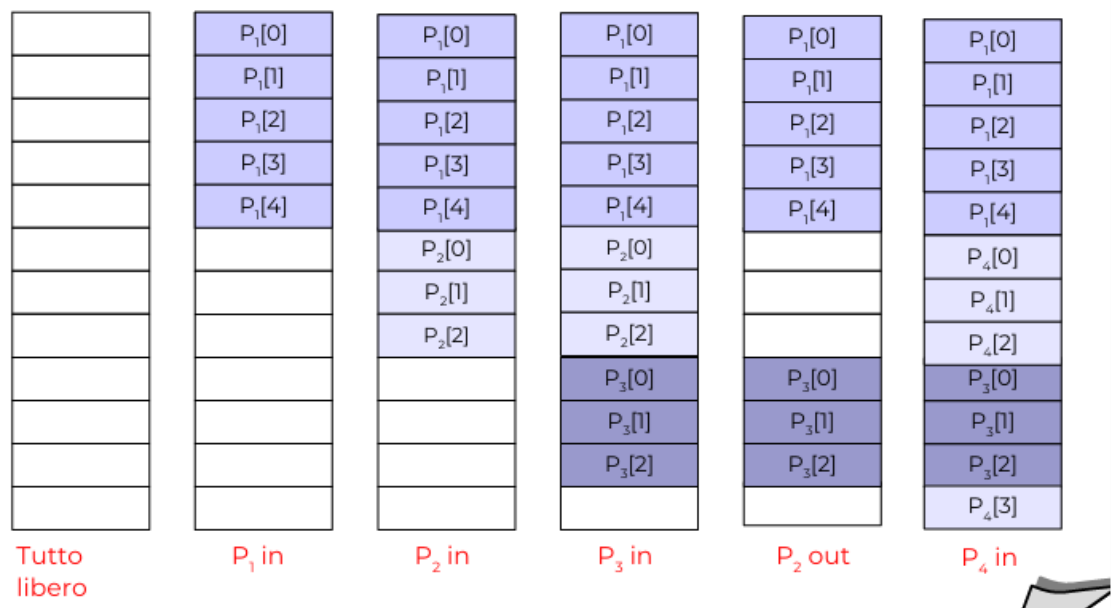


Figure 5.15: Paginazione con quattro processi P1, P2, P3, P4 che entrano ed escono. Le pagine di ogni processo vengono allocate nei frame liberi disponibili ovunque si trovino. Confrontato con la slide delle partizioni dinamiche, non si forma mai frammentazione esterna: i frame sono sempre completamente occupati o completamente liberi.

Questo esempio mostra il vantaggio cruciale della paginazione rispetto all'allocazione dinamica: quando P2 esce e P4 entra, le pagine di P4 vengono inserite nei frame lasciati liberi da P2, anche se non sono contigui a quelli di P3. Non esiste frammentazione esterna.

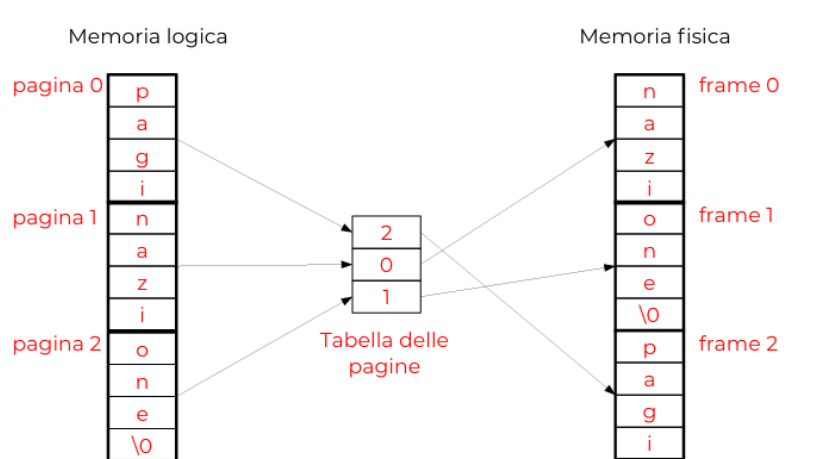


Figure 5.16: Corrispondenza tra memoria logica e fisica. La parola “paginazione” e’ divisa in tre pagine logiche (0, 1, 2). Le pagine logiche 0, 1, 2 vengono mappate rispettivamente nei frame fisici 2, 0, 1 (non contigui!). La page table tiene traccia di questa corrispondenza.

5.6.2 Traduzione degli Indirizzi con la MMU

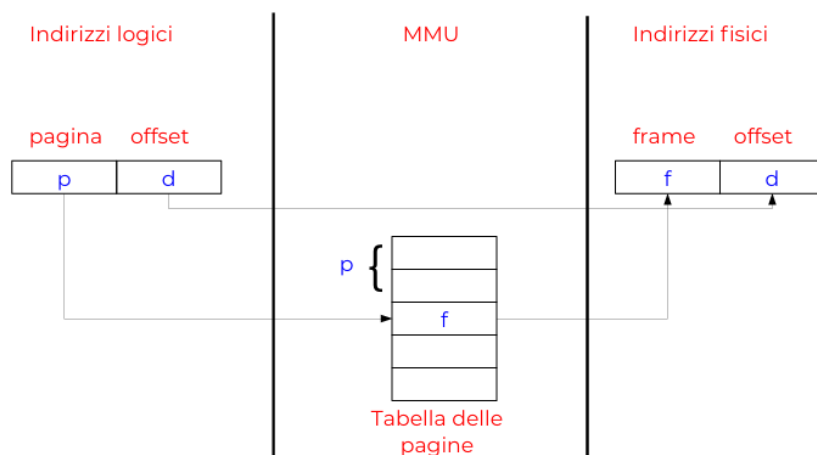


Figure 5.17: Schema hardware della MMU per la paginazione.

Un indirizzo logico a n bit viene interpretato come una coppia $\langle p, d \rangle$:

- p (i bit piu’ significativi): **numero di pagina** — indice nella page table del processo (es 52 bit).
- d (i bit meno significativi): **offset** all’interno della pagina — quanti byte dall’inizio della pagina (12 bit).

La page table contiene, per ogni pagina del processo, il numero del frame fisico f a cui e’ mappata. L’indirizzo fisico e’ $f \cdot (\text{dim. frame}) + d$, o equivalentemente $\langle f, d \rangle$ (concatenazione binaria, poiche’ la dimensione del frame e’ una potenza di 2).

5.6.3 Dimensione delle Pagine: il Trade-off

La dimensione delle pagine deve essere una **potenza di 2** per semplificare la traduzione (shift e maschere bit, senza divisioni).

	Pagine piccole	Pagine grandi
Frammentazione interna	Minima	Elevata (fino a meta' pagina)
Dimensione page table	Grande (2^{n-k} voci)	Piccola
Efficienza trasferimento I/O	Bassa (troppi trasferimenti)	Alta
TLB coverage	Basso (copre poca RAM)	Alto

Valori tipici: **4 KB** per le pagine normali; **2 MB** e **1 GB** per le *huge pages* usate da database e applicazioni HPC.

5.6.4 Implementazione della Page Table

Soluzione 1: Contenuta in Registri dedicati nella MMU (o anche CPU).
Troppo costoso: con pagine da 4 KB e spazio a 32 bit ci sono $2^{20} \approx 1$ milione di voci.

Soluzione 2: Tutta in RAM. La page table risiede in RAM; un registro speciale (**PTBR**) punta alla sua base. **Problema:** ogni accesso in memoria si raddoppia (un accesso per leggere la page table, uno per il dato).

Soluzione adottata: TLB(Translation lookaside buffer) + page table in RAM. La TLB e' una cache associativa che evita il doppio accesso nel caso tipico.

5.6.5 Translation Lookaside Buffer (TLB)

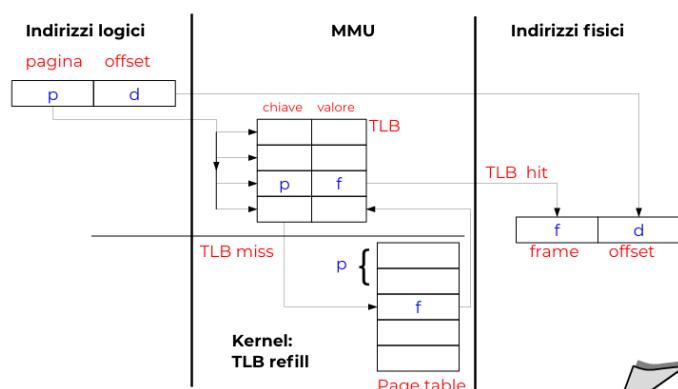


Figure 5.18: Schema completo MMU con TLB. Il numero di pagina p (chiave di ricerca) viene cercato in parallelo in tutte le voci del TLB. In caso di *hit* (freccia in alto), il frame f e' disponibile immediatamente: si compone l'indirizzo fisico. In caso di *miss* (freccia in basso), il kernel legge la page table in RAM (TLB refill), inserisce la nuova coppia nel TLB, e poi accede al dato.

Definizione – TLB — Translation Lookaside Buffer

Il **TLB** e' una piccola memoria cache associativa (*content-addressable memory*, CAM) integrata nella MMU. Contiene le traduzioni $\langle p \mapsto f \rangle$ piu' recentemente usate. Il lookup e' **parallelo**: il numero di pagina viene confrontato simultaneamente con tutte le chiavi del TLB in un unico ciclo di clock. Dimensioni tipiche: 8–2048 voci.

TLB Hit. p trovato nel TLB $\Rightarrow$ frame f restituito immediatamente. Costo totale: 1 accesso alla RAM (il dato). Nessun overhead rispetto a un sistema senza paginazione.

TLB Miss. p non trovato nel TLB $\Rightarrow$ il kernel legge la page table dalla RAM, ottiene f , inserisce $\langle p, f \rangle$ nel TLB, poi accede al dato. Costo: 2 accessi alla RAM (page table + dato).

Perche' funziona: il Principio di Localita'. I programmi accedono alle pagine con forte localita' temporale e spaziale: tendono a riusare le stesse pagine per lunghi periodi e ad accedere a pagine vicine. Un TLB con 64–512 voci copre tipicamente il 95–99% degli accessi (*TLB hit rate* molto alto), rendendo il costo medio della traduzione quasi nullo.

5.7 Segmentazione

Mentre la paginazione divide la memoria in blocchi di dimensione *fissa* in modo trasparente al programmatore, la **segmentazione** divide la memoria in aree di dimensione *variabile* con un significato *semantico*: ogni area (segmento) contiene elementi logicamente correlati.

Definizione – Segmento

Un **segmento** e' un'area di memoria logicamente continua che contiene elementi tra loro affini per tipo di accesso e permessi. Ogni segmento ha un **nome** (in pratica un indice numerico) e una **lunghezza**. Un riferimento di memoria e' dato dalla coppia $\langle s, d \rangle$: numero di segmento + offset.

Esempi di segmenti tipici in un programma C:

- **Text / Code**: istruzioni macchina. Permessi: sola lettura + esecuzione. Puo' essere **condiviso** tra piu' processi che eseguono lo stesso programma.
- **Data / BSS**: variabili globali e statiche. Lettura/scrittura; privato.
- **Stack**: variabili locali, indirizzi di ritorno. Lettura/scrittura; non condivisibile.
- **Heap**: memoria allocata dinamicamente (**malloc**). Lettura/scrittura; privato.

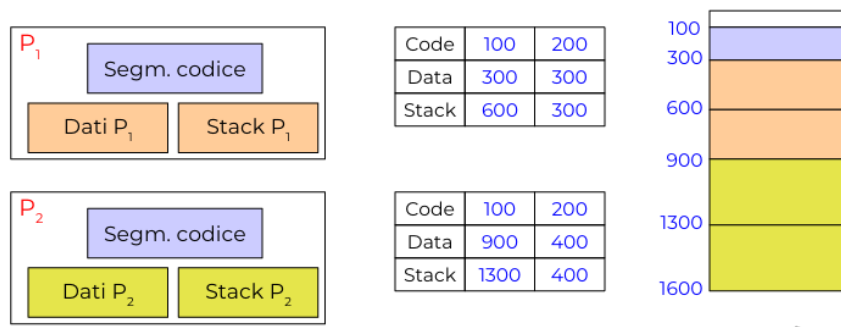


Figure 5.19: Condivisione del segmento di codice tra due processi P1 e P2. Il segmento “Code” (100–300 nella RAM fisica) e’ mappato nello spazio logico di entrambi i processi. I segmenti dati e stack rimangono privati e distinti. Questo risparmia la copia del codice del programma in RAM.

5.7.1 Segmentazione vs. Paginazione

Aspetto	Paginazione	Segmentazione
Chi decide la divisione	S.O. (automatica)	Compilatore/programmatore
Dimensione unita’	Fissa (es. 4 KB)	Variabile (KB – MB)
Contenuto	Eterogeneo	Omogeneo (solo codice, solo stack...)
Riferimento	Indirizzo numerico	Nome + offset
Frammentazione interna	Sì (ultima pagina)	No
Frammentazione esterna	No	Sì
Condivisione	Possibile	Naturale (per segmento)
Protezione	Per pagina	Per segmento (ideale)

5.8 Segmentazione + Paginazione

Allocare segmenti di dimensione variabile causa frammentazione esterna, come nell’allocazione dinamica. La soluzione pratica e’ combinare i due meccanismi:

- Lo spazio e’ diviso in **segmenti** (per condivisione e protezione).
- Ogni segmento e’ suddiviso in **pagine** che vengono allocate in frame liberi della memoria non necessariamente contigui (per eliminare la frammentazione esterna).

Si ottengono i benefici di entrambi: nessuna frammentazione esterna, protezione e condivisione per segmento. La MMU deve supportare entrambi gli stadi di traduzione (usato in x86-32 in modalita’ protetta).

5.9 Memoria Virtuale

Definizione – Memoria Virtuale

La **memoria virtuale** permette l'esecuzione di processi il cui spazio di indirizzamento *piu' grande* della RAM fisica, o comunque non e' completamente caricato in RAM. Ogni processo ha uno spazio di indirizzamento virtuale potenzialmente enorme (es. 2^{48} byte su x86-64) che la MMU mappa parzialmente sulla RAM fisica e parzialmente su disco.

E' importante notare che se viene implementata nel modo sbagliato può diminuire le prestazioni di un sistema (come vedremo nel paragrafo 5.7 riguardante la scelta degli algoritmi di rimpiazzamento)

5.9.1 Motivazione

Non e' necessario che *tutto* lo spazio di indirizzamento sia in RAM contemporaneamente perche':

- Le routine di gestione degli errori sono raramente invocate.
- Le strutture dati vengono dichiarate con dimensioni massime ma usate parzialmente.
- Un compilatore a due fasi usa memoria diversa in ciascuna fase.
- I branch condizionali fanno si' che solo parte del codice sia eseguita.

5.9.2 Implementazione Memoria Virtuale

- Ogni processo ha accesso ad uno spazio di indirizzamento virtuale che può essere più grande di quello fisico.
- Gli indirizzi virtuali (logici):
 - possono essere mappati su indirizzi fisici della memoria principale;
 - oppure, possono essere mappati su memoria secondaria.
- In caso di accesso ad indirizzi virtuali mappati in memoria secondaria:
 - i dati associati vengono trasferiti in memoria principale;
 - se la memoria è piena, si spostano in memoria secondaria i dati contenuti in memoria principale che sono considerati meno utili.

5.9.3 Implementazione: Demand Paging (Paginazione a Richiesta)

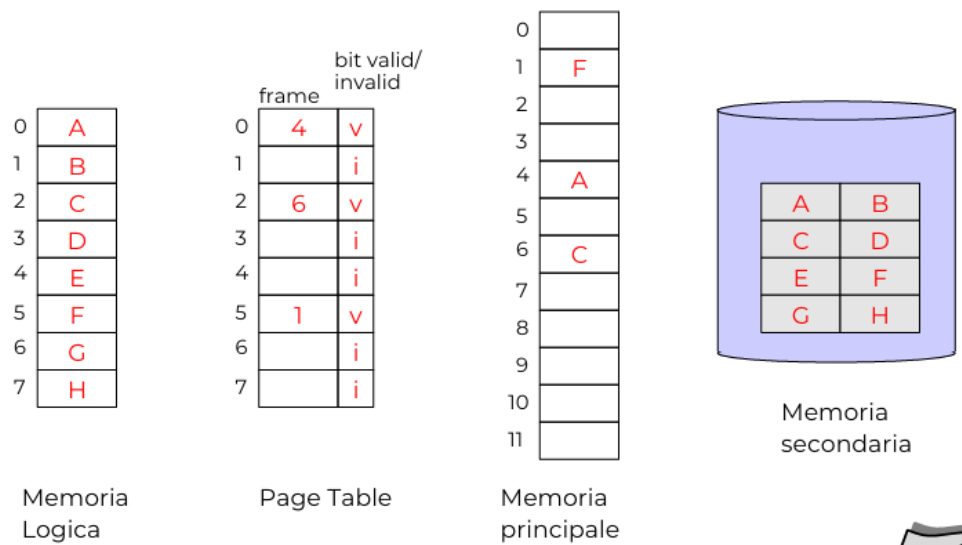


Figure 5.20: Stato della memoria con demand paging. La page table ha un bit v (valid): $v = 1$ significa che la pagina è in un frame fisico (il numero di frame è valido); $v = 0$ significa che la pagina è su disco (o non ancora allocata). Nell'esempio, le pagine A, C, F sono in memoria; le pagine B, D, E, G, H sono su disco.

Si utilizza la paginazione con una modifica: ogni voce della page table ha un **bit di validità** v :

- $v = 1$: pagina in memoria principale. Il numero di frame è valido.
- $v = 0$: pagina non in memoria (è su disco, nella *swap area*, o non ancora allocata).

Quando il processore accede a una pagina con $v = 0$, la MMU genera un **page fault** (trap hardware): il controllo passa al S.O. ed in particolare ad un suo componente **pager** che si occupa di caricare la pagina mancante in memoria, e inoltre aggiorna di conseguenza la tabella delle pagine

Gestione della Memoria: Swapping e Paginazione (approfondimento sulla terminologia usata)

Il concetto di Swap

Con il termine **swap** si intende l'azione di copiare interamente lo spazio di indirizzamento (l'area di memoria) utilizzato da un processo. Questa operazione si articola in due direzioni:

- **Swap-in:** il trasferimento dell'intero processo dalla memoria secondaria (disco) alla memoria principale (RAM).
- **Swap-out:** il trasferimento dell'intero processo dalla memoria principale alla memoria secondaria, solitamente per liberare spazio.

Si tratta di una tecnica storica, ampiamente utilizzata nei sistemi operativi del passato, prima che venisse introdotta la paginazione su richiesta.

Paginazione su richiesta (Demand Paging)

La **paginazione su richiesta** ha superato e sostituito lo swapping tradizionale di interi processi. Può essere vista concettualmente come una tecnica di swap di tipo *lazy* (pigro): il sistema non sposta più l'intero processo, ma **carica in memoria principale solo le pagine (i dati) che servono effettivamente** in un dato momento.

Evoluzione della terminologia: Pager e Swapper

Il passaggio dallo swapping alla paginazione ha lasciato alcune tracce nella terminologia dei sistemi operativi:

- **Swapper vs. Pager:** Alcuni sistemi operativi utilizzano ancora il termine *swapper* per indicare il *pager* (il componente che gestisce lo spostamento delle singole pagine). Oggi, riferirsi al pager con il termine "swapper" è considerata una terminologia **obsoleta**.
- **Swap Area:** Nonostante lo swapping di interi processi non sia più lo standard, il termine **swap area** (o area di swap) è rimasto in uso. Oggi viene utilizzato correttamente per indicare quella specifica porzione del disco (memoria secondaria) impiegata per ospitare le singole pagine rimosse temporaneamente dalla memoria principale.

5.9.4 Gestione Dettagliata di un Page Fault

Supponiamo inizialmente che il processo (in pagina 0) cerca di accedere alla pagina 1 (contenente i dati B) che si trova su disco.

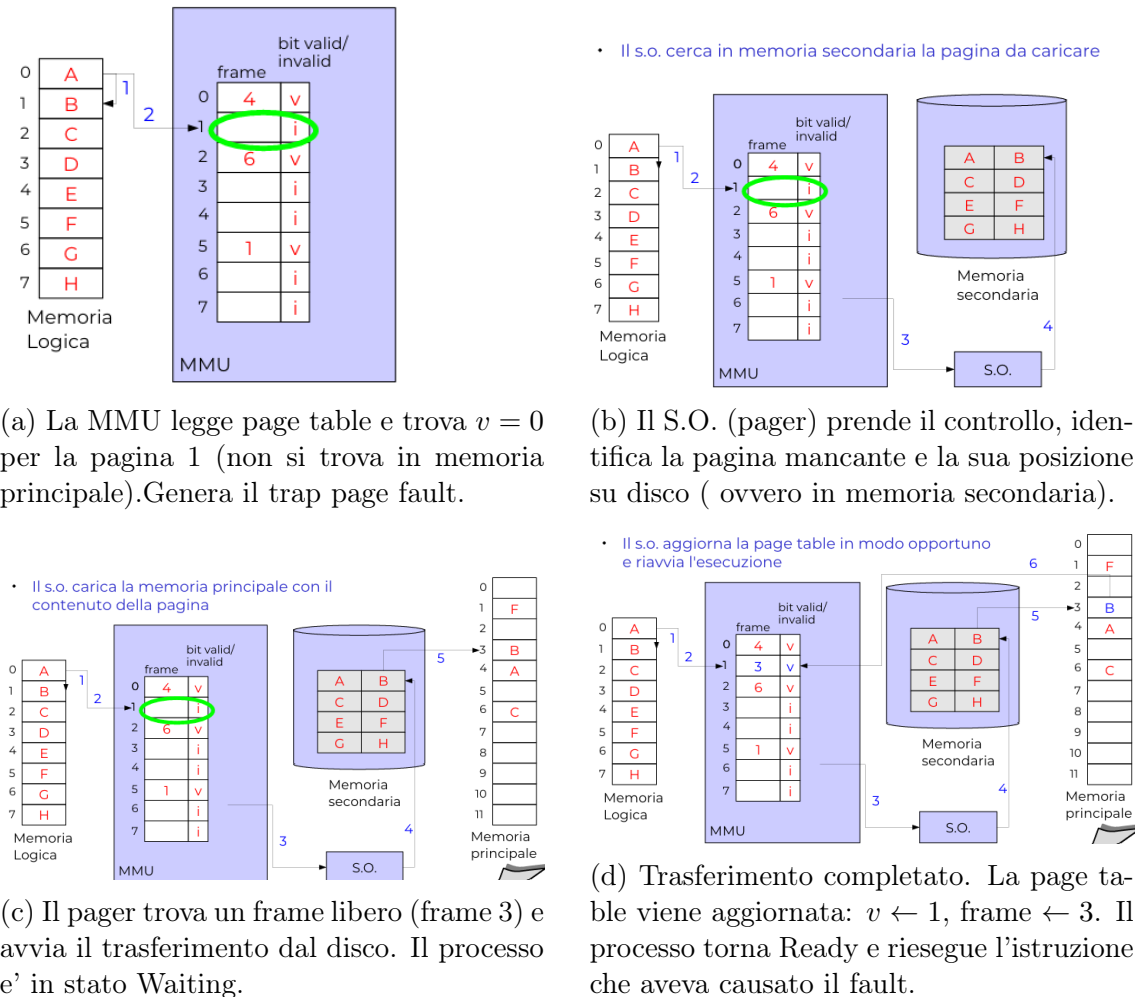


Figure 5.21: Le quattro fasi della gestione di un page fault. Da notare: il processo è bloccato solo durante il trasferimento I/O; nel frattempo la CPU può eseguire altri processi.

Algoritmo del meccanismo di demand paging

- Individua la pagina in memoria secondaria
- Individua un frame libero
- Se non esiste un frame libero
 - richiama algoritmo di rimpiazzamento
 - aggiorna la tabella delle pagine (invalida pagina "vittima")
 - se la pagina "vittima" è stata variata, scrive la pagina sul disco
 - aggiorna la tabella dei frame (frame libero)
- Aggiorna la tabella dei frame (frame occupato)
- Leggi la pagina da disco (quella che ha provocato il fault)
- Aggiorna la tabella delle pagine (caricato il TLB)
- Riattiva il processo

Figure 5.22: Algoritmo per il demanding paging che presenta anche l'algoritmo di rimpiazzamento spiegato nella sessione successiva [5.10](#)

Nota – Dirty Bit

Ogni voce della page table ha anche un **dirty bit** (o *modified bit*): viene settato a 1 dalla MMU ogni volta che si *scrive* nella pagina. In fase di rimpiazzamento, se la pagina vittima ha dirty bit = 0, può essere scartata direttamente (è identica alla copia su disco); se dirty bit = 1, deve essere salvata su disco prima di essere sovrascritta. Questo ottimizza il numero di scritture su disco.

5.10 Algoritmi di Rimpiazzamento

Quando un page fault si verifica senza frame liberi, il S.O. sceglie una **pagina vittima** da scaricare. L'algoritmo di rimpiazzamento determina quale pagina viene scelta, con l'obiettivo di minimizzare il numero futuro di page fault (ovvero eliminando la pagina meno utile).

Nota – Stringa di riferimenti

Gli algoritmi (ideati per minimizzare il numero di page fault) vengono valutati su una **stringa di riferimenti**: la sequenza di numeri di pagina acceduti nel tempo.

Esempio: stringa completa `71,0a,13,25,0a,3f,...` con pagine da 16 byte diventa la stringa ridotta `7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1` (si tiene solo la cifra esadecimale piu' significativa, che indica il numero di pagina).

aspettative algoritmo di rimpiazzamento

Algoritmi di rimpiazzamento

- **Andamento dei page fault in funzione del numero di frame**
 - ci si aspetta un grafo monotono decrescente...
 - ma non sempre è così

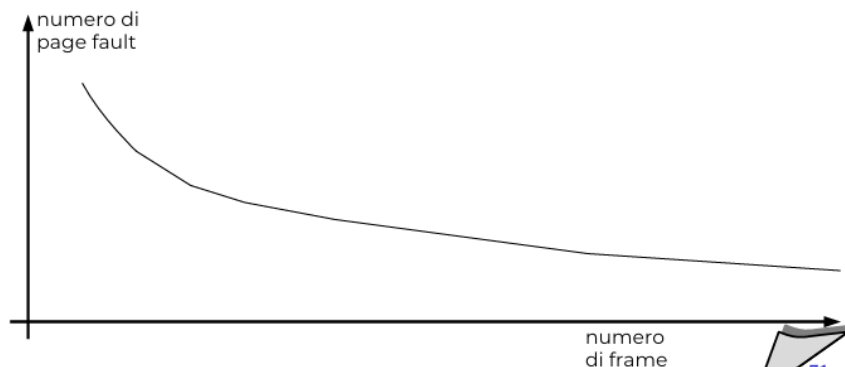


Figure 5.23

5.10.1 Algoritmo FIFO

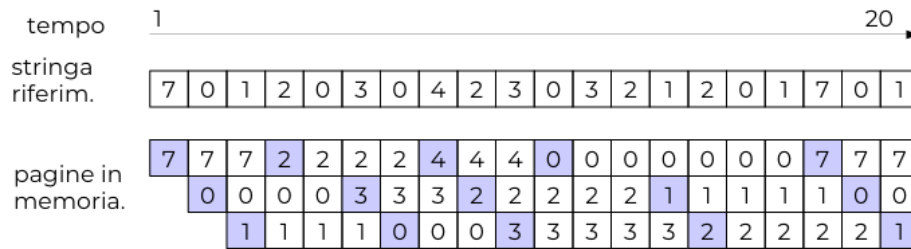


Figure 5.24: FIFO con 3 frame sulla stringa di 20 riferimenti: 15 page fault (quadrati azzurri nelle colonne). Le tre righe mostrano il contenuto dei tre frame nel tempo; . Si noti come la pagina 0, acceduta spesso, venga scaricata (causando fault evitabili) solo perché “vecchia”.

Il **First-In, First-Out** è il più semplice: la vittima è sempre la pagina **caricata in memoria per prima**. Si mantiene una coda FIFO dei frame; la testa è la vittima.

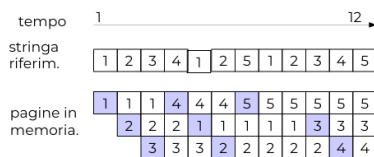
Vantaggi: semplicissimo; non richiede alcun supporto hardware.

Svantaggi: non distingue tra pagine usate di frequente (“calde”) e pagine inattive (“fredde”); può scaricare pagine molto usate solo perché caricate da tempo.

Anomalia di Belady

• Caratteristiche

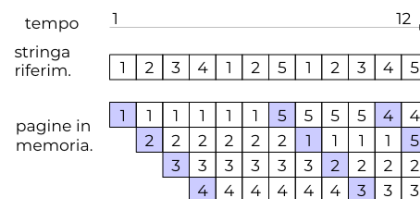
- numero di frame in memoria: 3
- numero di page fault: 9 (su 12 accessi in memoria)



(a) FIFO con 3 frame sulla stringa 1 2 3 4 1 2 5 1 2 3 4 5: 9 page fault.

• Caratteristiche

- numero di frame in memoria: 4
- numero di page fault: 10! (su 12 accessi in memoria)
- il numero di page fault è aumentato!



(b) FIFO con 4 frame sulla *stessa* stringa: 10 page fault! Un frame in più, un fault in più.

- In alcuni algoritmi di rimpiazzamento:
 - non è detto che aumentando il numero di frame il numero di page fault diminuisca (e.g., FIFO)
- Questo fenomeno indesiderato si chiama **Anomalia di Belady**



Figure 5.26: Grafico dell'Anomalia di Belady: il numero di page fault non è monotono decrescente all'aumentare dei frame. Il picco a 4 frame è chiaramente visibile.

Definizione – Anomalia di Belady

L'**Anomalia di Belady** è il fenomeno per cui certi algoritmi (come FIFO) possono generare *più* page fault con *più* frame. Avviene perché con un frame in più cambia quale pagina viene sacrificata, e la nuova scelta può essere paradossalmente peggiore per la sequenza futura di accessi.

La spiegazione intuitiva: con 4 frame, le pagine 1 e 2 vengono mantenute in memoria più a lungo ma questo causa la perdita di 3, 4 ecc. in momenti meno opportuni rispetto al caso con 3 frame.

5.10.2 Algoritmo Ottimale (MIN / OPT)

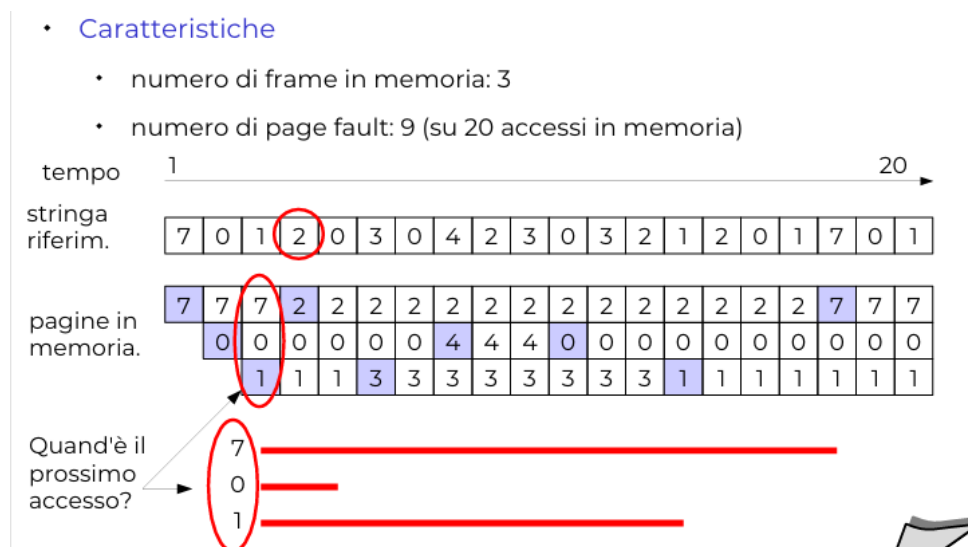


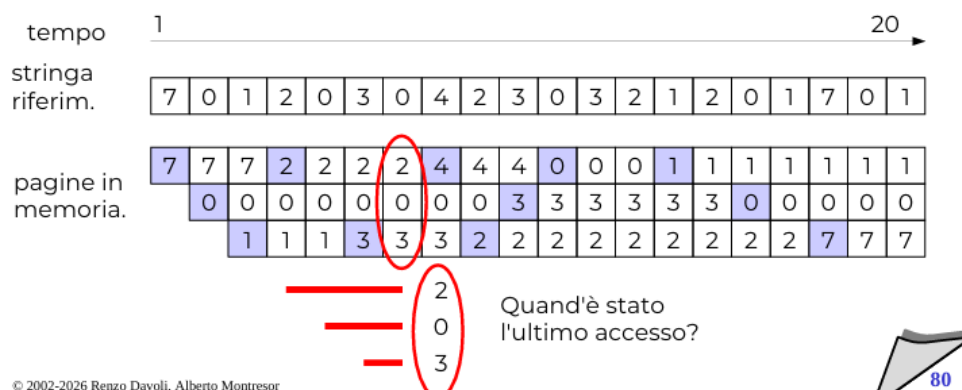
Figure 5.27: Algoritmo MIN con 3 frame: solo 9 page fault su 20 accessi (contro i 15 del FIFO). La vittima è sempre la pagina il cui prossimo accesso è più lontano nel futuro. Le linee rosse mostrano quando ciascuna pagina sarà nuovamente acceduta.

L'algoritmo **MIN** (o **OPT**) sceglie come vittima la pagina che non verrà più acceduta oppure quella il cui prossimo accesso è il più lontano nel futuro. Produce il **minimo numero possibile di page fault** per qualunque stringa e numero di frame, ma non è implementabile (richiederebbe la conoscenza del futuro).

Serve come **limite inferiore teorico** per valutare gli algoritmi reali.

5.10.3 Algoritmo LRU (Least Recently Used)

- **Caratteristiche**
 - numero di frame in memoria: 3
 - numero di page fault: 12 (su 20 accessi in memoria)



© 2002-2026 Renzo Davoli, Alberto Montresor



Figure 5.28: LRU con 3 frame: 12 page fault su 20 accessi. La vittima è la pagina acceduta meno recentemente. Per ogni colonna, la pagina in basso è la prossima vittima (quella il cui “ultimo accesso” è il più vecchio).

Definizione – LRU — Least Recently Used

La vittima e' la pagina che non viene acceduta da *piu' tempo*. Si basa sul **principio di localita'**: se una pagina e' stata usata di recente e' probabile che lo sia ancora presto; se non viene acceduta da molto, probabilmente non servira' a breve. LRU approssima MIN usando la distanza nel *passato* come stima della distanza nel *futuro*.

Perche' LRU funziona bene: il principio di localita'

LRU si giustifica teoricamente grazie al **principio di localita' temporale**: i programmi reali tendono ad accedere ripetutamente alle stesse pagine in finestre di tempo ristrette (cicli, funzioni, strutture dati "calde"). Questo principio non e' un'assunzione arbitraria, bensì una proprieta' empiricamente osservabile in quasi tutti i programmi.

La scelta della vittima di LRU puo' essere vista come l'inverso di MIN:

- MIN conosce il futuro ed elimina la pagina usata piu' lontano nel *futuro*;
- LRU non conosce il futuro, ma usa la distanza nel *passato* come approssimazione ragionevole.

Nota – LRU vs. MIN vs. FIFO — confronto sul numero di page fault

- **MIN**: 9 page fault (ottimale, non implementabile in pratica).
- **LRU**: 12 page fault (buona approssimazione di MIN).
- **FIFO**: 15 page fault (peggiore, non sfrutta la localita').

LRU e' il compromesso migliore tra efficienza e implementabilita'. Non soffre dell'anomalia di Belady (vedremo perche' nella Sezione 5.10.3).

Implementazione Necessita di uno specifico supporto Hardware**1 - Implementazione con Timestamp**

L'implementazione piu' diretta associa a ogni voce della page table un **contatore logico** (timestamp):

1. La CPU (o la MMU) mantiene un registro contatore globale, incrementato a ogni riferimento in memoria.
2. Ogni volta che si accede a una pagina, il contatore corrente viene copiato nella voce corrispondente della page table.
3. Quando si deve scegliere la vittima, si scorre l'intera page table e si seleziona la voce con il **contatore minore** (timestamp piu' vecchio).

Nota – Problemi dell'implementazione con timestamp

- **Overflow del contatore**: se il contatore e' a 32 bit e ci sono miliardi di accessi al secondo, va in overflow. Occorre gestire il wraparound.
- **Ricerca lineare**: trovare la pagina con il timestamp minore richiede di

scorrere tutta la page table, con costo $O(n)$ dove n e' il numero di frame. Con molti frame, questo e' costoso.

- **Supporto hardware:** richiede che la MMU aggiorni il timestamp a *ogni accesso*, non solo ai page fault. Questo ha un costo non trascurabile.

2 - Implementazione con Stack (Double-Linked List)

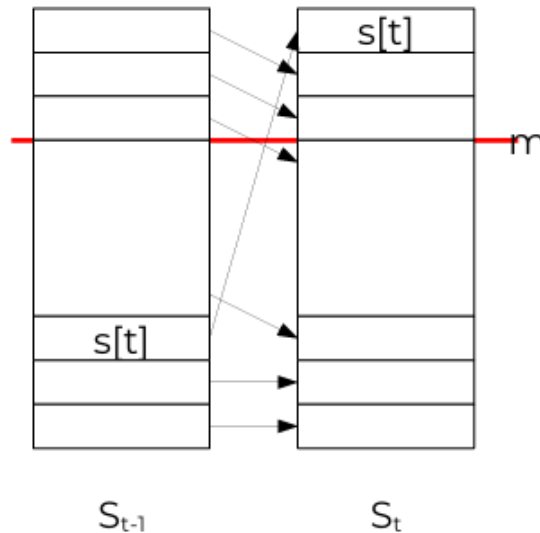


Figure 5.29: LRU come algoritmo a stack. Ad ogni riferimento $s[t]$, la pagina acceduta viene spostata in cima allo stack (facendo scorrere le successive). Le prime m pagine in cima (insieme S_t con m frame) sono sempre un sottoinsieme delle prime $m + 1$ (S_t con $m + 1$ frame): questa e' la proprieta' "a stack".

L'implementazione con **lista doppiamente collegata** risolve il problema della ricerca lineare della vittima:

1. Le pagine in memoria sono organizzate in una lista doppiamente collegata, ordinata per recenza di accesso: la **testa** contiene la pagina acceduta piu' di recente, la **coda** contiene la pagina acceduta meno di recente (la vittima LRU).
2. Ad ogni accesso alla pagina p :
 - a. Se p e' gia' nella lista: la si rimuove dalla posizione corrente (aggiornamento di 4 puntatori).
 - b. Si inserisce p in testa alla lista (aggiornamento di 2 puntatori).
3. La **vittima** e' sempre il nodo in coda: si accede in $O(1)$.

In totale, ogni accesso comporta al piu' **6 modifiche a puntatori**, con costo $O(1)$ sia per la ricerca della vittima sia per l'aggiornamento.

Esempio – Aggiornamento della lista LRU

Supponiamo di avere 4 frame con la lista (dalla piu' recente alla meno recente): $[A \rightarrow B \rightarrow C \rightarrow D]$. Arriva un accesso alla pagina C :

1. Rimuoviamo C dalla posizione attuale: $[A \rightarrow B \rightarrow D]$
2. Inseriamo C in testa: $[C \rightarrow A \rightarrow B \rightarrow D]$

La nuova vittima e' D (la pagina acceduta meno di recente). Se ora arriva un accesso a una pagina nuova E (page fault), espelliamo D e inseriamo E in testa: $[E \rightarrow C \rightarrow A \rightarrow B]$.

Nota – Problema pratico: supporto hardware

Anche l'implementazione a lista richiede che l'**hardware aggiorni la struttura dati ad ogni accesso in memoria**, inclusi gli accessi normali (non solo i page fault). Questo e' irrealistico su architetture reali: la RAM viene acceduta miliardi di volte al secondo, e aggiornare una lista collegata ad ogni accesso saturerebbe il bus dati.

Per questo motivo, l'LRU *esatto* non viene mai implementato nei sistemi operativi reali: si usano invece le **approssimazioni** descritte nella sezione sui reference bit, che si basano su un singolo bit hardware.

Algoritmi a Stack**Definizione – Algoritmo a Stack**

Un algoritmo di rimpiazzamento A e' detto **a stack** se per ogni:

- stringa di riferimenti s ,
- istante t
- quantità di memoria m :

$$S_t(s, A, m) \subseteq S_t(s, A, m + 1)$$

dove $S_t(s, A, m)$ e' l'insieme delle pagine in memoria all'istante t con m frame. Con un frame in più si mantengono *almeno* le stesse pagine con m frame, piu' eventualmente una aggiuntiva.

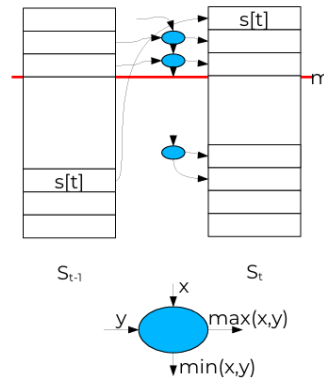


Figure 5.30: Struttura generale degli algoritmi a stack. Ogni riferimento $s[t]$ porta la pagina acceduta in cima. La relazione d'ordine tra le pagine (chi è più o meno “importante”) dipende solo dalla stringa di riferimenti e dal tempo corrente, non dal numero di frame. Questo garantisce che aggiungere un frame non possa mai peggiorare la situazione.

Perché la proprietà a stack impedisce l'anomalia di Belady

Definizione

Teorema: un algoritmo a stack non può mai generare l'anomalia di Belady, cioè con più frame non può avere più page fault che con meno frame.

Dimostrazione: Sia $s[t+1]$ il prossimo riferimento dopo l'istante t . Si considerino i tre casi possibili:

1. s si trova nell'intersezione:

$$s[t+1] \in S_t(m)$$

la pagina è già in memoria sia con m che con $m+1$ frame (per la proprietà a stack). **Nessun page fault** in entrambi i casi.

2. s non appartiene a nessuno dei due:

$$s[t+1] \notin S_t(m+1)$$

la pagina non è in memoria con $m+1$ frame, quindi non è neanche in $S_t(m)$ (perché $S_t(m) \subseteq S_t(m+1)$). **Page fault in entrambi i casi.**

3. $s[t+1] \in S_t(m+1)$ ma $s[t+1] \notin S_t(m)$: la pagina è in memoria con $m+1$ frame ma non con m . **Page fault solo con m frame**, non con $m+1$. Aggiungere un frame *aiuta*.

In nessun caso aggiungere frame provoca page fault aggiuntivi. □

Nota – Quali algoritmi sono a stack?

- **LRU:** sì, è a stack. Le m pagine con m frame sono sempre un sottoinsieme delle $m+1$ pagine con $m+1$ frame (si veda la struttura della lista).
- **MIN (OPT):** sì, è a stack.
- **FIFO:** no. L'anomalia di Belady è la prova diretta che FIFO non è a

stack: con 4 frame puo' avere piu' fault che con 3.

3 - Approssimazioni di LRU con il Reference Bit

Come detto, LRU esatto e' troppo costoso da implementare in hardware. Tutti i processori moderni forniscono pero' un supporto minimo: il **reference bit** (detto anche *accessed bit* in alcune architetture).

Definizione – Reference Bit

Ogni voce della page table contiene un **reference bit** (bit R) inizialmente tutti a zero:

- La **MMU** lo pone a 1 automaticamente ogni volta che si accede alla pagina (in lettura o scrittura), senza nessun intervento del software.
- Il **sistema operativo** puo' leggerlo e resettarlo (0) in qualsiasi momento.

Il reference bit fornisce un'informazione binaria grossolana: “questa pagina e' stata usata dall'ultimo reset?”.

NOTA: Non dice *quante* volte ne' *quando* esattamente, ma e' sufficiente per costruire buone approssimazioni di LRU.

Esistono 2 particolari algoritmi per il reference bit:

a) Additional Reference Bits Algorithm



Figure 5.31: Additional Reference Bits. La pagina con il valore binario minore e' la vittima (quella usata meno di recente).

Questo algoritmo estende il singolo reference bit a una **storia recente** di n bit per pagina (tipicamente $n = 8$).

Meccanismo: a intervalli regolari (es. ogni 100 ms), il timer interrupt scatena la seguente operazione su tutte le pagine:

1. Il registro di storia viene **shiftato a destra** di 1 bit.
2. Il **reference bit corrente** viene inserito come MSB (bit piu' significativo).

Esempio – Lettura della storia a 8 bit

Supponiamo $n = 8$ e tre pagine con storie:

- Pagina A : 11001100 — usata spesso, anche di recente.
- Pagina B : 00000001 — usata molto tempo fa, non di recente.
- Pagina C : 01110111 — usata frequentemente nel passato recente.

Interpretando i byte come interi senza segno: $A = 204$, $B = 1$, $C = 119$. La vittima e' **B** (valore minore): e' la pagina che non viene acceduta da piu' tempo.

- **Pro:** Approssima bene LRU con soli n bit extra per pagina. La **risoluzione temporale** è definita dall'intervallo del timer.
- **Contro:** In caso di parità (stesso valore binario), si usa FIFO come spareggio. La precisione dipende dalla **frequenza degli shift**.

b) Second-Chance Algorithm (Algoritmo dell'Orologio)

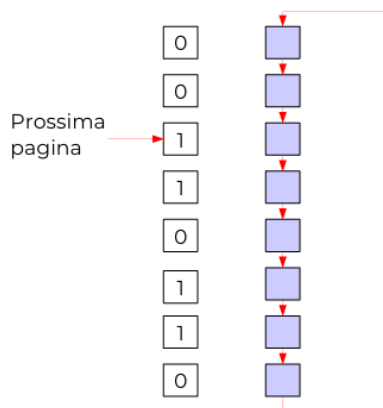


Figure 5.32: Second-Chance (Clock). Le pagine formano un anello circolare. La lancetta avanza: reference bit 1 → azzeralo e avanza (“seconda possibilità”); reference bit 0 → questa è la vittima. Nell’esempio la lancetta si ferma alla terza pagina incontrata con bit a 0 (vedi figura 5.33).

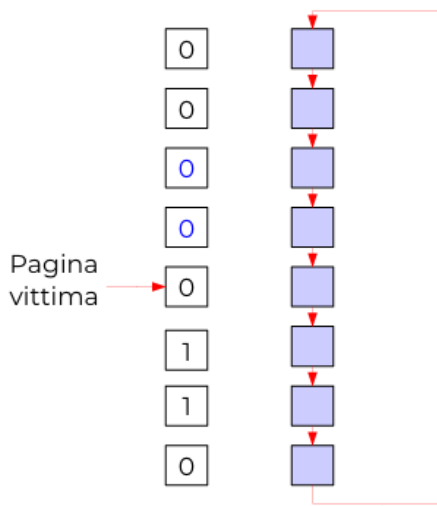


Figure 5.33: Esecuzione concreta del clock: partendo dalla freccia, le prime due pagine incontrate hanno bit 1 (vengono azzerate e risparmiate), la terza ha bit 0 ed è la vittima (evidenziata).

Il **Second-Chance** (o *Clock Algorithm*) è l'approssimazione di LRU più usata nella pratica. È di fatto una versione di FIFO con una “seconda chance” per le pagine usate di recente.

Struttura dati: le pagine in memoria sono organizzate in un **buffer circolare** (anello), con un puntatore “lancetta” che avanza ciclicamente.

Algoritmo (eseguito a ogni page fault per trovare la vittima):

1. Esamina la pagina puntata dalla lancetta.
2. Se il reference bit e' 1: azzeralo, avanza la lancetta. (La pagina e' stata usata di recente: le diamo una seconda possibilita' di rimanere in memoria.)
3. Se il reference bit e' 0: **questa e' la vittima**. Rimuovila (scrivendola su disco se e' dirty), inserisci la nuova pagina in questa posizione, avanza la lancetta. Vi e' quindi una selezione in modo FIFO

Esempio – Traccia di esecuzione del Clock

Buffer circolare con 5 frame, lancetta punta alla posizione 1:

Posizione	Pagina	Ref bit
1 (lancetta →)	<i>A</i>	1
2	<i>B</i>	1
3	<i>C</i>	0
4	<i>D</i>	1
5	<i>E</i>	0

Scansione:

- Pos. 1, *A*, bit=1: azzerà → 0, avanza.
- Pos. 2, *B*, bit=1: azzerà → 0, avanza.
- Pos. 3, *C*, bit=0: **vittima**. *C* viene espulsa.

La lancetta resta in posizione 3 (dove viene inserita la nuova pagina).

Caso degenerare: se tutte le pagine hanno reference bit = 1, la lancetta compie un giro completo azzerandoli tutti, dopodiché sceglie la prima pagina incontrata (comportamento identico a FIFO). In pratica questo accade solo quando il sistema e' gravemente a corto di frame (thrashing).

Nota – Perché il Clock e' così usato nella pratica

- Richiede solo **1 bit per pagina** (già fornito dall'hardware).
- Struttura dati: un semplice **array circolare** con un indice intero. Nessuna lista collegata da aggiornare ad ogni accesso.
- Costo per trovare la vittima: $O(n)$ nel caso peggiore (giro completo), ma ammortizzato e' $O(1)$ in condizioni normali.
- Approssima LRU ragionevolmente bene senza nessun supporto hardware aggiuntivo oltre al reference bit.

Linux usa una variante di questo algoritmo nella *page cache* (con due liste, *active* e *inactive*, che approssimano due livelli di “recency”).

5.10.4 Least Frequently Used (LFU)

Definizione – LFU — Least Frequently Used

LFU mantiene per ogni pagina un **contatore di accessi** totali. La vittima e' la pagina con il contatore piu' basso (quella usata meno frequentemente dall'inizio della sua permanenza in memoria).

L'intuizione e' diversa da LRU: invece di privilegiare le pagine accedute *di recente*, LFU privilegia quelle accedute *piu' spesso*. In alcuni workload questa puo' essere un'approssimazione migliore.

Problema fondamentale: una pagina molto usata nella *fase di inizializzazione* del programma accumula un contatore alto che non riflette piu' il suo utilizzo attuale. Rimarrà in memoria anche se non viene piu' acceduta, “rubando” frame a pagine attive.

Soluzione parziale — Aging: il contatore viene **dimezzato periodicamente** (right-shift di 1 bit a ogni intervallo di tempo). Questo introduce un “decadimento esponenziale” che riduce progressivamente il peso degli accessi passati, rendendo il contatore piu' sensibile agli accessi recenti.

Table 5.1: Riepilogo degli algoritmi di rimpiazzamento delle pagine.

Algoritmo	Principio	Implementabile?	Anomalia lady?	Be-
MIN/OPT	Sostituisce la pagina usata piu' lontano nel futuro	No (richiede futuro)	No (e' a stack)	
FIFO	Sostituisce la pagina caricata in memoria da piu' tempo	Si', banale	Si'	
LRU	Sostituisce la pagina acceduta meno di recente	Costoso in hardware	No (e' a stack)	
Clock	Approssima LRU con reference bit	Si', economico	No (approssima LRU)	
LFU (+aging)	Sostituisce la pagina acceduta meno spesso	Si'	No	

5.11 Allocazione dei Frame, Thrashing e Working Set

5.11.1 Strategie di Allocazione dei Frame (memoria virtuale)

Si intende l'algoritmo utilizzato per scegliere quanti frame assegnare ad ogni singolo processo. Ne esistono di due tipi:

- **Allocazione locale.** Ogni processo ha un numero fisso di frame riservati. Per rimpiazzare, sceglie tra le sue stesse pagine. Prevedibile ma rigido.
- **Allocazione globale.** Tutti i processi competono per tutti i frame del sistema. Flessibile ma può causare thrashing.

5.11.2 Thrashing

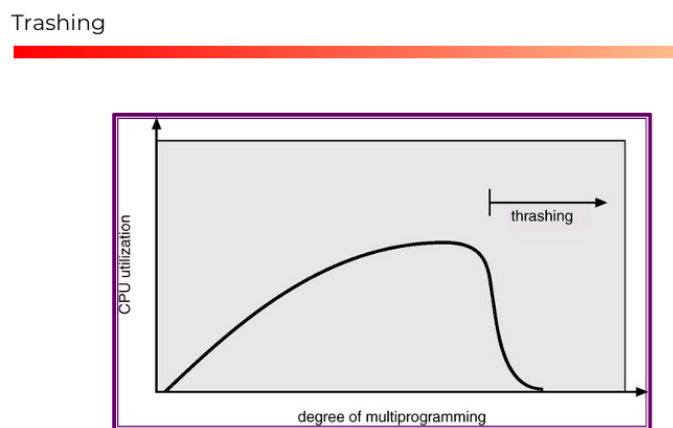


Figure 5.34: Il thrashing: la curva di utilizzo della CPU in funzione del grado di multiprogrammazione. Inizialmente sale (più processi = migliore utilizzo). Raggiunto un picco, crolla drammaticamente: il sistema passa quasi tutto il tempo a gestire page fault invece di eseguire codice utile.

Definizione – Thrashing

Un processo (o un sistema) è in **thrashing** quando spende più tempo a gestire page fault (I/O su disco, ovvero paginazione) che a eseguire istruzioni utili (esecuzione).

Sintomo: utilizzo della CPU basso nonostante tutti i processi siano “attivi”; I/O del disco al massimo; sistema apparentemente bloccato.

Nota

Cause possibili

- I processi si rubano a vicenda i frame
- Non si riescono a tenere in memoria frame utili a breve termine, in quanto altri processi richiedono frame liberi, generando così page fault ogni pochi passi di avanzamento

Meccanismo di innesco con allocazione globale:

1. Molti processi $\Rightarrow$ ogni processo ha pochi frame $\Rightarrow$ page fault continui.
2. I processi vanno in Waiting per I/O $\Rightarrow$ la ready queue si svuota.
3. Lo scheduler vede CPU sotto-utilizzata $\Rightarrow$ carica nuovi processi (ERRORE!).
4. Più processi $\Rightarrow$ meno frame per ognuno $\Rightarrow$ ancora più page fault.
5. Il ciclo si avvita: il sistema satura il disco con I/O di swap.

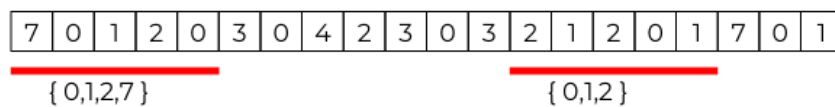
5.11.3 Working Set

Figure 5.35: Il Working Set con finestra $\Delta = 5$. In figura è mostrata la stringa di riferimenti; sotto, le linee rosse indicano i confini della finestra scorrevole. Al tempo mostrato nella prima linea il WS è $\{0, 1, 2, 7\}$; alla seconda è $\{0, 1, 2\}$. Il WS cambia dinamicamente seguendo la località del processo.

Definizione – Working Set

Il **working set** di finestra Δ al tempo t è l'insieme delle pagine accedute negli ultimi Δ riferimenti:

$$WS(t, \Delta) = \{\text{pagina acceduta al tempo } \tau \mid t - \Delta < \tau \leq t\}$$

È un'approssimazione della **località corrente** del processo: l'insieme di pagine che gli servono “adesso”.

Serve quindi a stimare di quante pagine ha bisogno un processo in determinati istanti

Uso pratico. Sia $|WS_i|$ la dimensione del working set del processo i . La condizione di sicurezza è:

$$\sum_i |WS_i| \leq F_{\text{totale (disponibili)}}$$

- Se $\sum |WS_i| > F_{\text{totale}} \Rightarrow$ il sistema è in (o vicino al) thrashing: si deve sospendere (*swap-out*) almeno un processo per liberare frame.
- Se $\sum |WS_i| \ll F_{\text{totale}} \Rightarrow$ ci sono frame inutilizzati: si può attivare un nuovo processo.

Scelta di Δ :

- Δ troppo piccolo $\Rightarrow$ sottostima del WS, non considerando più utile ciò che in realtà serve $\Rightarrow$ troppi page fault (falsi negativi di thrashing).

- Δ troppo grande $\Rightarrow$ sovrastima del WS, considerando utile anche ciò che non serve più $\Rightarrow$ frame sprecati, meno processi attivi (falsi positivi di thrashing).
- Δ ideale: abbraccia esattamente la fase di esecuzione corrente (una funzione, un ciclo, un passo dell'algoritmo).

Riepilogo del Capitolo

Meccanismo	Framm. interna	Framm. esterna	Rilocazione
Partizioni fisse	Alta	No	No
Partizioni dinamiche	No	Si'	Si' (compattazione)
Paginazione	Bassa	No	Si'
Segmentazione	No	Si'	Si'
Seg. + Paginazione	Bassa	No	Si'
Memoria virtuale	Bassa	No	Si'

1. **Binding**: associazione indirizzi logici $\rightarrow$ fisici. Compile-time (semplice, no multiprogrammazione), load-time (rilocalizzazione statica), run-time (MMU, flessibilità massima).
2. **MMU**: traduce indirizzi logici in fisici in hardware. La versione base usa registro di rilocalizzazione + limite; la versione avanzata usa la page table con TLB.
3. **TLB**: cache associativa per le traduzioni più recenti. Funziona grazie al principio di località. Al context switch va invalidata.
4. **Allocazione contigua**: partizioni fisse (semplice, framm. interna) e dinamiche (flessibile, framm. esterna). Algoritmi di selezione: First Fit migliore in pratica.
5. **Paginazione**: elimina framm. esterna. Page table mappa pagine logiche su frame fisici non contigui.
6. **Segmentazione**: aree con significato semantico. Condivisione e protezione naturali, ma framm. esterna.
7. **Memoria virtuale** con demand paging: le pagine assenti generano page fault e vengono caricate dal disco. Dirty bit ottimizza le scritture su disco.
8. **Algoritmi di rimpiazzamento**: FIFO (semplice, anomalia di Belady), MIN (ottimale teorico), LRU (ottimo pratico, a stack), Clock (approssimazione efficiente di LRU).
9. **Working Set e Thrashing**: il WS misura la località corrente; mantenere $\sum |WS_i| \leq F_{\text{totale}}$ previene il thrashing.